

ARC42 · SOFTWAREARCHITEKTUR-DOKUMENTATION

Personal- & Warenwirtschaftssystem

arc42-Architekturdokumentation

Version 1.0 · 6. September 2026

Basierend auf der arc42-Vorlage (Version 8, deutsch) · arc42.de,
Lizenz CC BY-SA

arc42-Architekturdokumentation: Personal- & Warenwirtschaftssystem

Basierend auf der [arc42](#)-Vorlage (Version 8, deutsch). Lizenz arc42: CC BY-SA.

1. Einführung und Ziele

1.1 Aufgabenstellung

Das System digitalisiert zwei Kernprozesse eines filialisierten Einzelhandelsunternehmens (Vorbild: Lidl-artige Lebensmittel-/Discounterkette mit mehreren Filialen):

Personalmanagement

- Erfassung von Arbeitszeiten durch Mitarbeiter (Kommen/Gehen, Pausen)
- Beantragung und Genehmigung von Zeitkorrekturen (z. B. vergessene Stempelung)
- Einsicht und Pflege des eigenen Mitarbeiterprofils (Stammdaten, Verträge, Stundenübersicht)
- Filial- und HR-seitige Verwaltung von Mitarbeiterdaten

Warenwirtschaft

- Inventur (Soll-/Ist-Abgleich von Lagerbeständen je Filiale)
- Abschreibung von Waren (Verderb, Bruch, Diebstahl) mit Begründung und Nachvollziehbarkeit
- Preisabfrage von Artikeln (z. B. per Barcode) am Verkaufsort
- Pflege von Artikelstammdaten und Lagerbeständen je Filiale

Ziel ist die Ablösung papier- bzw. tabellenbasierter Prozesse durch eine zentrale, rollenbasierte Webanwendung, die in Echtzeit Transparenz für Mitarbeiter, Filialleiter, HR und Geschäftsführung schafft.

1.2 Qualitätsziele

Rang	Qualitätsziel	Motivation
1	Sicherheit	Personaldaten (Arbeitszeiten, Kontaktdaten, ggf. Gehaltsbezüge) sind besonders schützenswert (DSGVO). Zugriff muss strikt rollenbasiert erfolgen.
2	Nachvollziehbarkeit / Auditierbarkeit	Korrekturen an Arbeitszeiten und Warenbestands-Abschreibungen sind lohn- bzw. bilanzrelevant und müssen lückenlos nachvollziehbar sein (wer hat wann was geändert und warum).
3	Benutzbarkeit	Endnutzer sind Verkaufs-/Lagerpersonal ohne IT-Hintergrund, Bedienung erfolgt ggf. an Kassen-Terminals oder Tablets zwischen Kundenbedienungen — muss einfach und schnell sein.

Rang	Qualitätsziel	Motivation
4	Verfügbarkeit	Zeiterfassung und Preisabfrage werden täglich im Filialbetrieb benötigt; Ausfälle stören den operativen Ablauf unmittelbar.
5	Wartbarkeit / Erweiterbarkeit	Weitere Module (z. B. Urlaubsverwaltung, Bestellwesen) sind absehbar; die Architektur muss neue fachliche Bausteine ohne große Umbauten aufnehmen können.

1.3 Stakeholder

Rolle	Erwartungshaltung
Mitarbeiter	Einfache, schnelle Zeiterfassung; Einsicht in eigenes Profil und geleistete Stunden
Filialleiter	Übersicht/Korrektur der Arbeitszeiten seiner Filiale; Durchführung von Inventuren und Abschreibungen
HR / Personalabteilung	Zentrale, filialübergreifende Sicht auf Mitarbeiterdaten und Arbeitszeiten; Basis für Lohnabrechnung
Admin / IT	Verwaltung von Benutzerkonten, Rollen, Filialen; Systembetrieb
Geschäftsführung	Aggregierte Kennzahlen (Personalkosten, Schwund/Abschreibungen)
Entwickler	Wartbare, gut dokumentierte, testbare Architektur (Portfolio-Projekt mit Fokus auf sauberem Systemdesign)

2. Randbedingungen

Organisatorisch

- Einzelentwickler-Projekt (Portfolio) mit begrenzter Zeit — Architektur muss von einer Person umsetzbar und wartbar bleiben.
- Deutschsprachige Zielgruppe (Filialpersonal in DACH-Region); UI-Texte primär Deutsch.
- Iterative Entwicklung: MVP zuerst (Zeiterfassung + Basis-Warenwirtschaft), Ausbau danach.

Technisch (vorgegeben / bereits etabliert)

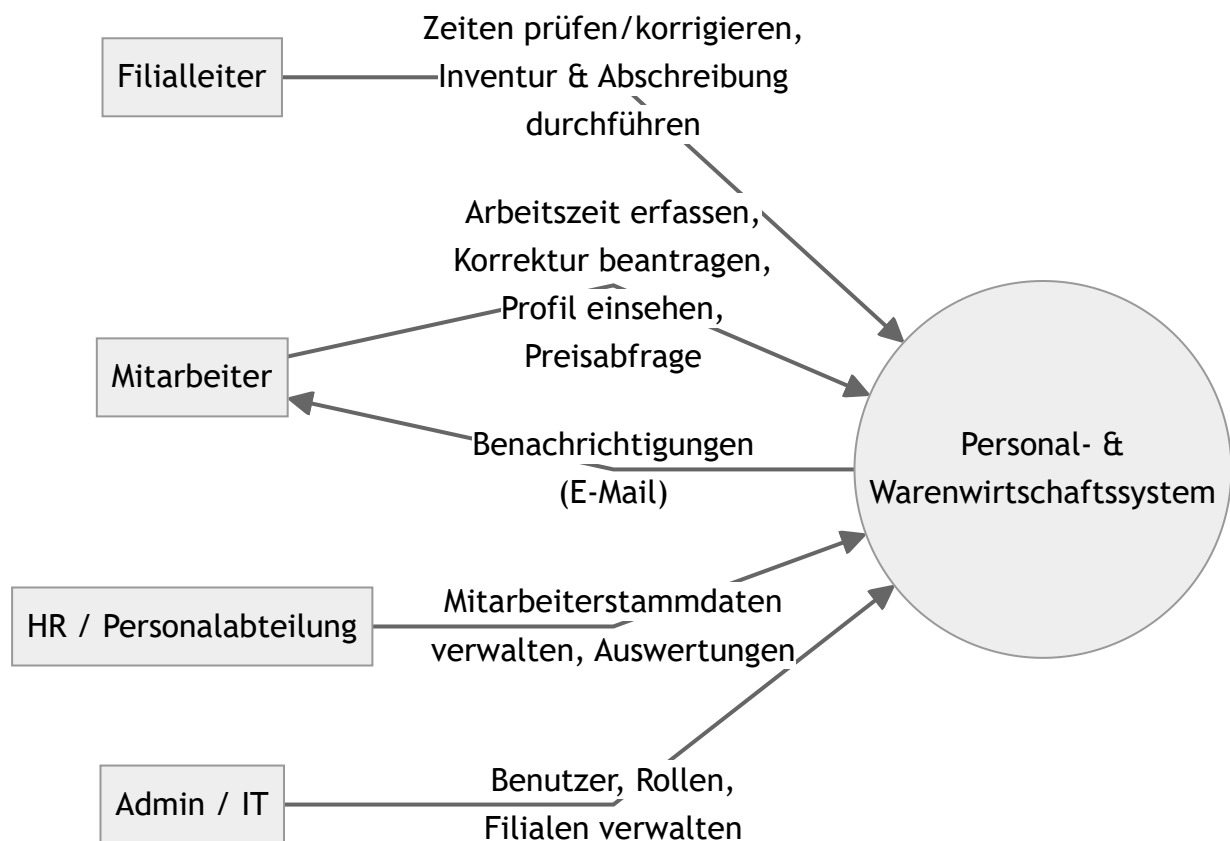
Bereich	Vorgabe
Backend	Java 17, Spring Boot 4.1.1 (Spring Web MVC, Spring Data JPA, Spring Validation), Maven
Datenbank	PostgreSQL, Schema-Migrationen via Flyway
Frontend	Angular 22 (Standalone Components), TypeScript, Vitest für Tests
Versionierung	Git-Monorepo (backend/ , frontend/), Branches main (stabil) / development
Deployment-Ziel	Cloud-nativ, Kubernetes-Cluster (siehe Kapitel 7)
Auth	Eigenes Login mit JWT (kein externer IdP)

Rechtlich / Konventionen

- DSGVO: Verarbeitung personenbezogener Daten (Arbeitszeiten, Stammdaten) erfordert Löschkonzept, Zweckbindung und Zugriffsprotokollierung.
- REST/JSON als API-Stil, HTTPS verpflichtend in allen Umgebungen außer lokaler Entwicklung.

3. Kontextabgrenzung

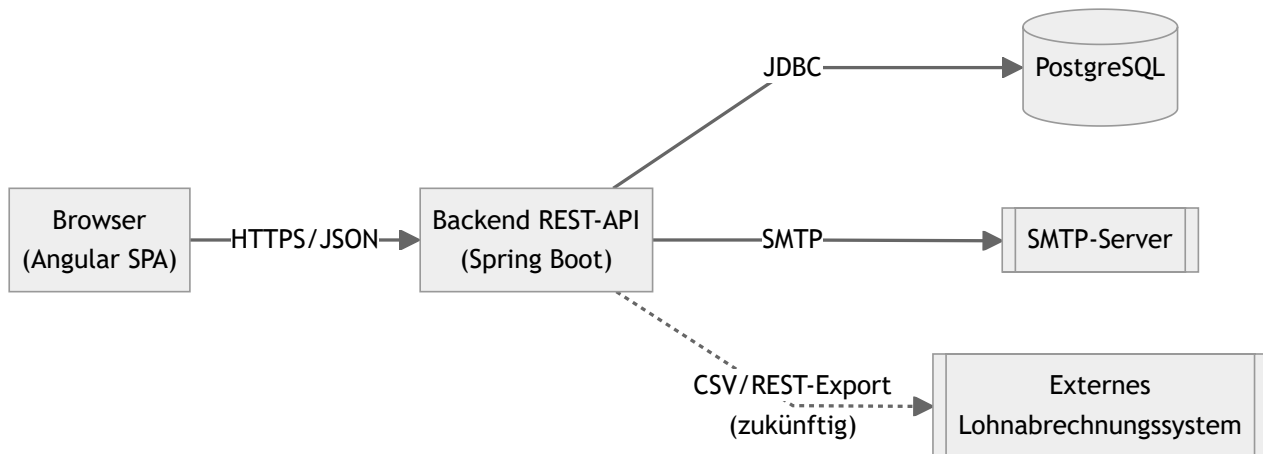
3.1 Fachlicher Kontext



Fachliche Nachbarsysteme (aktuell nicht Teil des MVP, aber als künftige Schnittstellen berücksichtigt):

Nachbarsystem	Richtung	Zweck
Lohnabrechnungssystem	Ausgehend	Export freigegebener Arbeitszeiten je Abrechnungsperiode
E-Mail-Server	Ausgehend	Benachrichtigungen (Korrektur genehmigt/abgelehnt, Passwort-Reset)
Kassensystem / Barcode-Scanner	Eingehend	Artikel-Identifikation für Preisabfrage und Inventur

3.2 Technischer Kontext



4. Lösungsstrategie

Thema	Entscheidung	Begründung
Architekturstil	Modularer Monolith (kein Microservices-Schnitt)	Überschaubare Domäne, ein Entwickler, geringerer Betriebsaufwand; klare Modulgrenzen erlauben spätere Auftrennung bei Bedarf
Fachliche Gliederung	Bounded Contexts Personalmanagement, Warenwirtschaft, Stammdaten/Kern (Firma, Filiale, Benutzer)	Spiegelt die zwei fachlichen Hauptdomänen aus der Aufgabenstellung; Stammdaten werden von beiden genutzt
Datenhaltung	PostgreSQL, ein Schema, Flyway-Migrationen	ACID-Garantien für Arbeitszeiten und Lagerbestände wichtig; Flyway sorgt für nachvollziehbare, versionierte Schemaentwicklung
Frontend-Architektur	Angular SPA mit rollenbasierter Navigation/Guards	Eine Codebasis für alle Rollen, unterschiedliche Ansichten je nach Berechtigung
Authentifizierung	Zustandsloses JWT (Access + Refresh Token), Spring Security	Skaliert horizontal ohne Session-Replikation — passend zu Kubernetes-Deployment mit mehreren Pod-Replicas
Autorisierung	Rollenbasiert (RBAC): Mitarbeiter, Filialleiter, HR, Admin	Deckt die in Kapitel 1.3 identifizierten Stakeholder-Rollen ab
Mandantenmodell	Eine Firma, mehrere Filialen (kein Multi-Tenant)	Anforderung: interne Lösung für ein Unternehmen, nicht SaaS für mehrere Kundenfirmen
Deployment	Containerisiert, Kubernetes-Cluster	Zukunftsfähig, horizontale Skalierung des Backends bei Lastspitzen (z. B. Monatswechsel/Inventur)

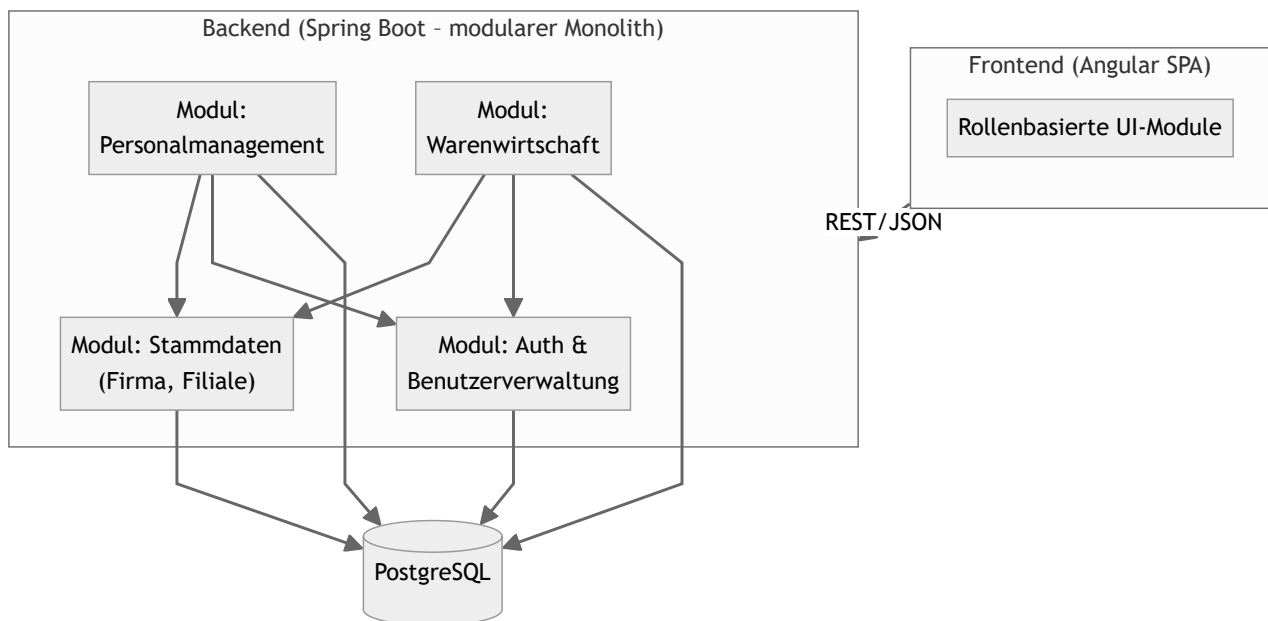
Erreichung der Qualitätsziele (Kurzfassung):

- *Sicherheit*: JWT + RBAC + HTTPS + Input-Validierung (Details Kapitel 8)
- *Nachvollziehbarkeit*: Audit-Trail für Korrekturen/Abschreibungen (Details Kapitel 8)
- *Benutzbarkeit*: Fokussierte, rollenspezifische Angular-Views, mobile-taugliches Layout

- *Verfügbarkeit:* Mehrere Backend-Replicas hinter Load Balancer, Health-Checks (Details Kapitel 7)
- *Wartbarkeit:* Modulare Paketstruktur nach Bounded Context statt nach technischer Schicht

5. Bausteinsicht

5.1 Whitebox Gesamtsystem



Baustein	Verantwortung
Auth & Benutzerverwaltung	Login, Token-Ausstellung/-Erneuerung, Passwort-Verwaltung, Rollenzuordnung
Stammdaten	Firma- und Filialverwaltung; Referenzdaten, die von beiden Fachmodulen genutzt werden
Personalmanagement	Zeiterfassung, Zeitkorrekturanträge inkl. Genehmigungsworkflow, Mitarbeiterprofil
Warenwirtschaft	Artikelstammdaten, Lagerbestand je Filiale, Inventur, Abschreibung, Preisabfrage

5.2 Ebene 2 — Modul-Innensicht

Modul Auth & Benutzerverwaltung

- `AuthController` — Login, Token-Refresh, Logout
- `JwtService` — Token-Erstellung/-Validierung, Signierung
- `BenutzerService` / `BenutzerRepository` — Verwaltung von `Benutzer`, `Rolle`
- Spring-Security-Filterkette (`JwtAuthenticationFilter`) zur Absicherung aller Endpunkte

Modul Personalmanagement

- `ZeiterfassungController` — Kommen/Gehen buchen, eigene Einträge abrufen

- `ZeitkorrekturService` — Korrekturantrag stellen, genehmigen/ablehnen (Statemachine: `OFFEN` → `GENEHMIGT/ABGELEHNT`)
- `ProfilController` — eigenes Profil ansehen/bearbeiten (eingeschränkt), HR-Sicht auf alle Profile
- Entities: `Mitarbeiter` , `Arbeitszeiteintrag` , `Korrekturantrag`

Modul Warenwirtschaft

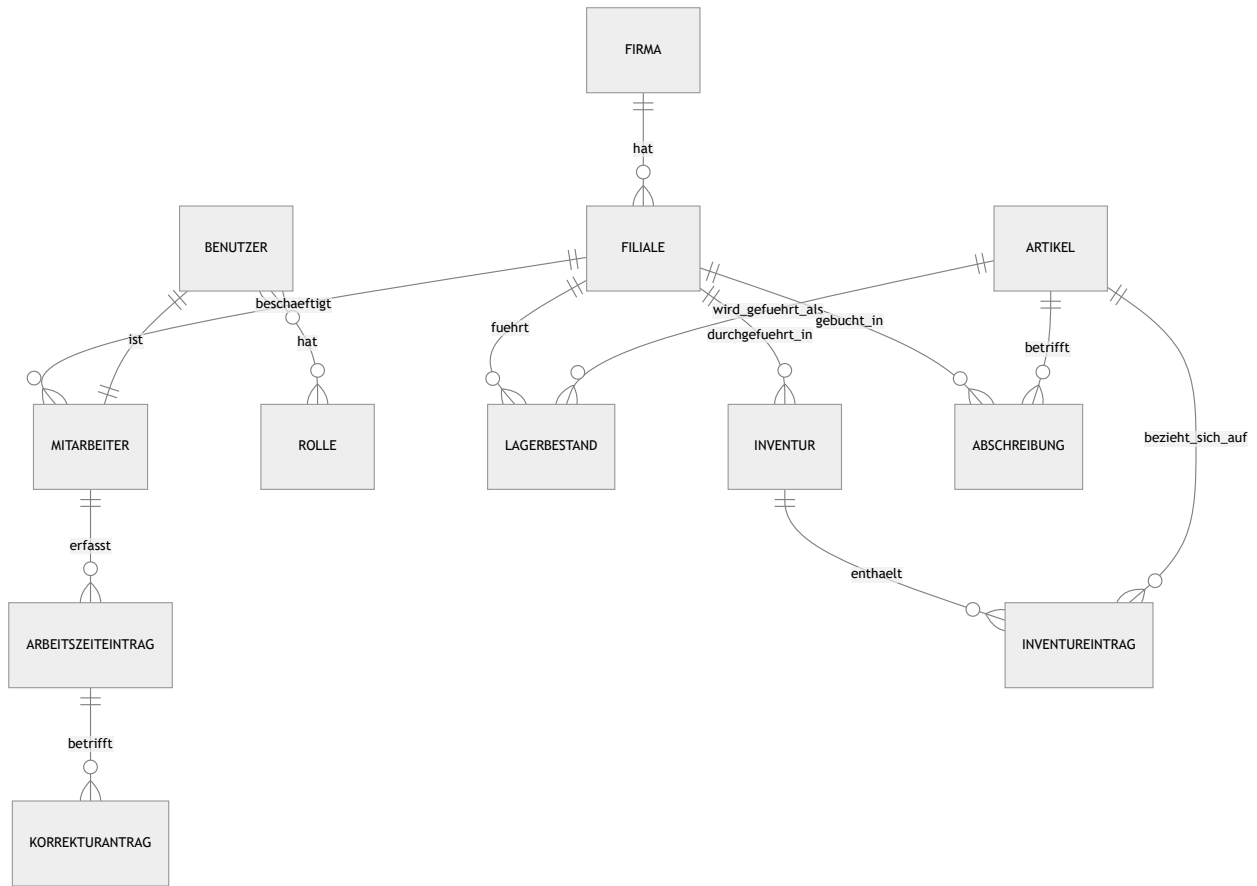
- `ArtikelController` — Artikelstammdaten (CRUD, primär Admin/HR-nah bzw. zentrale Einkaufsrolle)
- `PreisabfrageController` — schneller Read-Endpunkt nach Barcode/Artikelnummer
- `InventurService` — Inventur starten, Ist-Bestände erfassen, Differenz zu Soll-Bestand berechnen
- `AbschreibungService` — Abschreibung mit Grund (Verderb/Bruch/Diebstahl/Sonstiges), Mengen- und Wertermittlung
- Entities: `Artikel` , `Lagerbestand` (je Filiale), `InventurEintrag` , `Abschreibung`

Modul Stammdaten

- Entities: `Firma` , `Filiale`
- Einfache CRUD-Services, primär von Admin genutzt

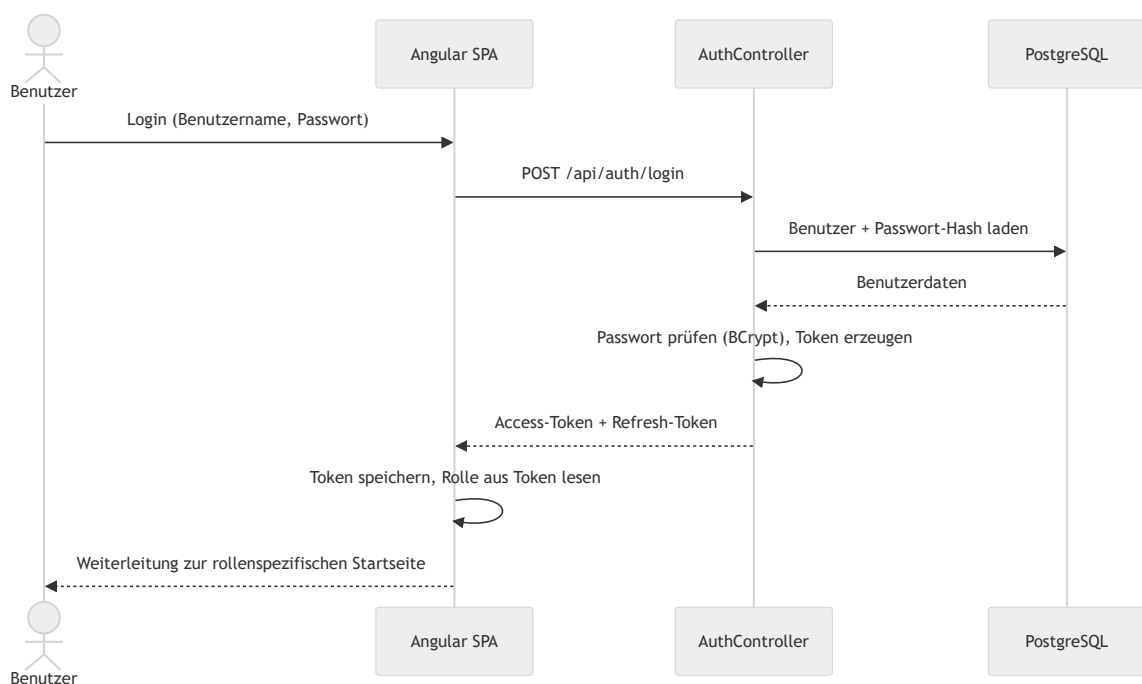
Innerhalb jedes Moduls gilt eine einheitliche Schichtung: **Controller** → **Service** → **Repository (Spring Data JPA)** → **Datenbank**, DTOs an der Controller-Grenze zur Entkopplung von Entities und API-Verträgen.

5.3 Wichtige Entitätsbeziehungen (Kern-Datenmodell, vereinfacht)

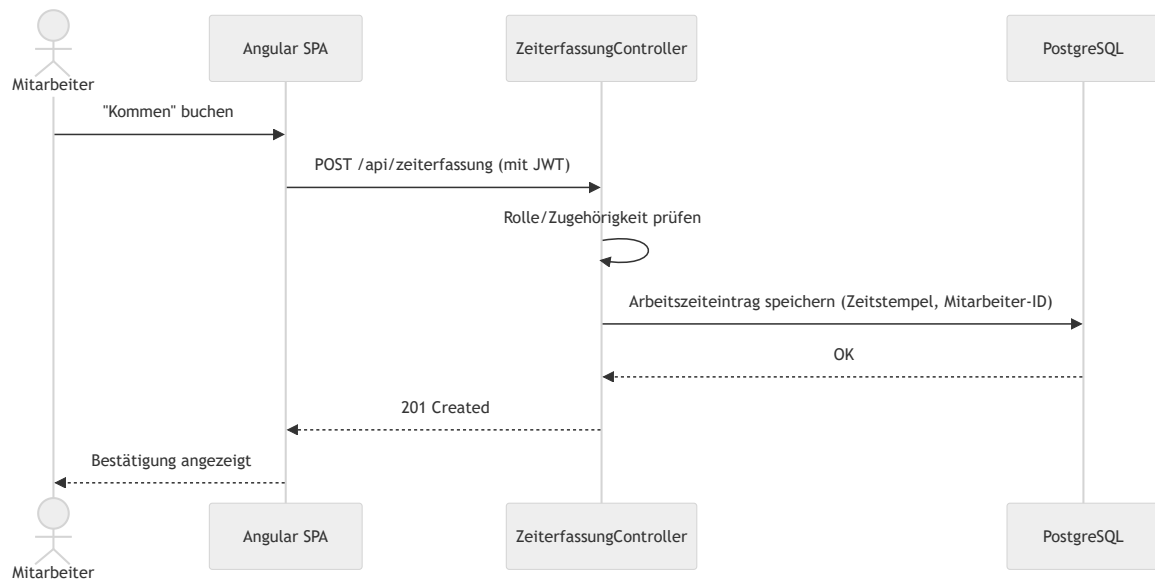


6. Laufzeitsicht

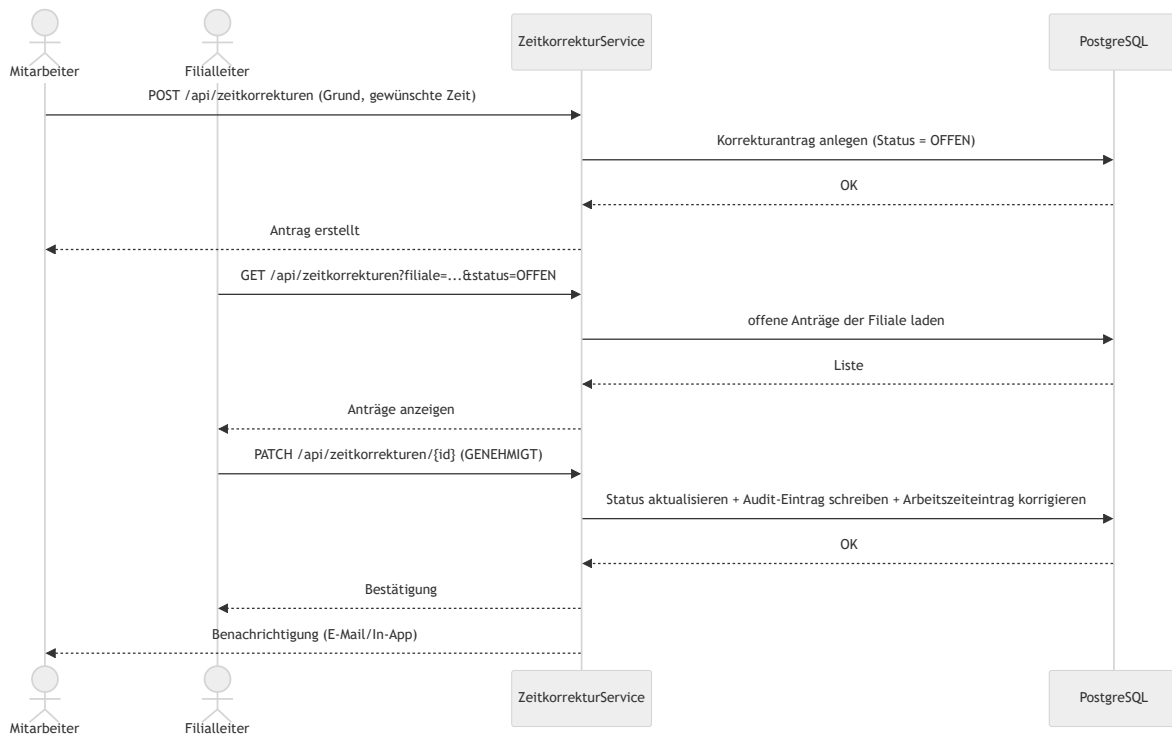
6.1 Login mit JWT



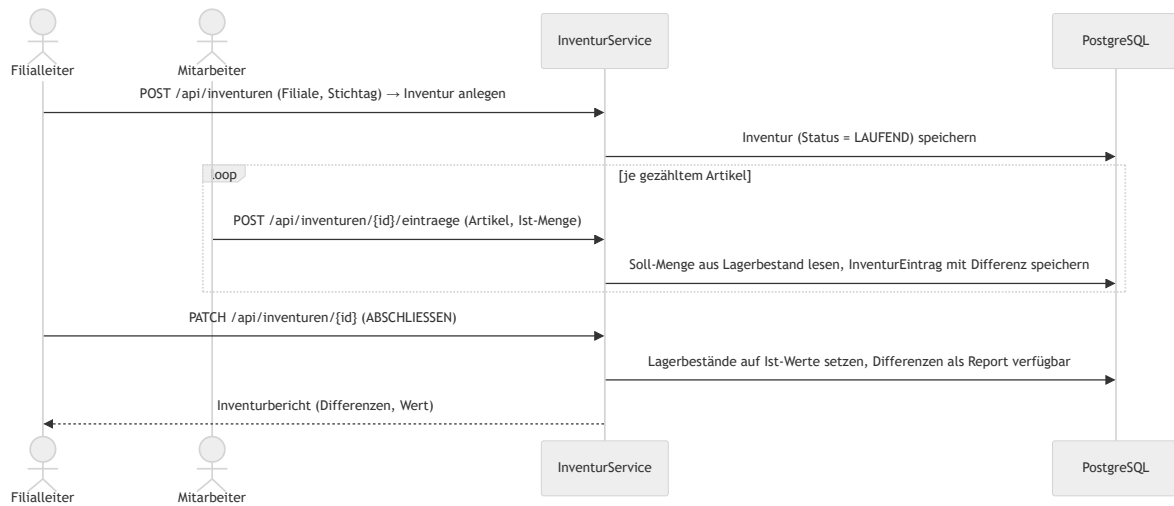
6.2 Arbeitszeit erfassen



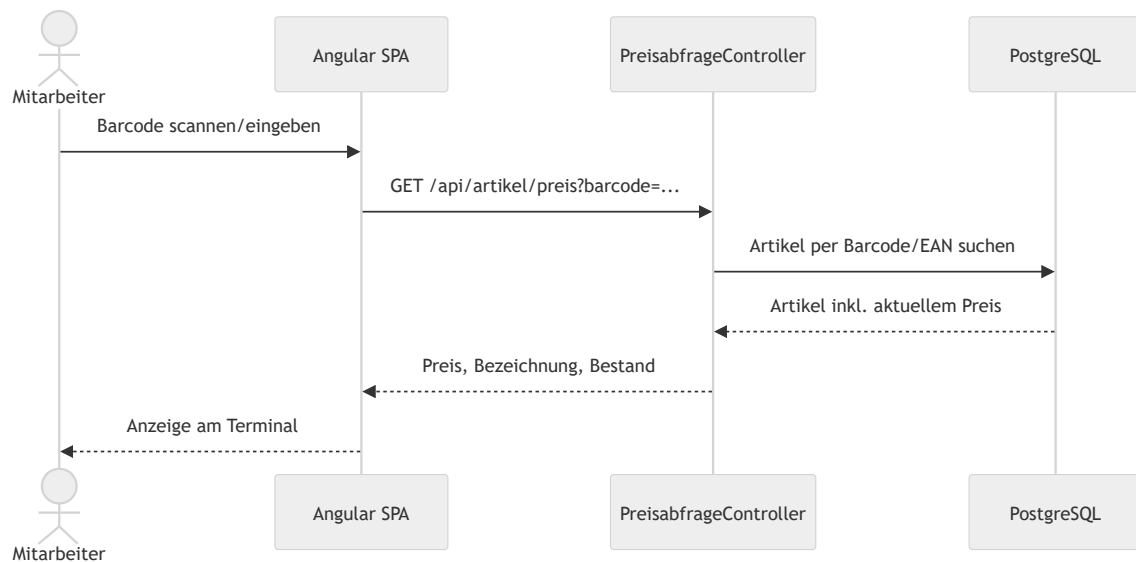
6.3 Zeitkorrektur beantragen und genehmigen



6.4 Inventur durchführen

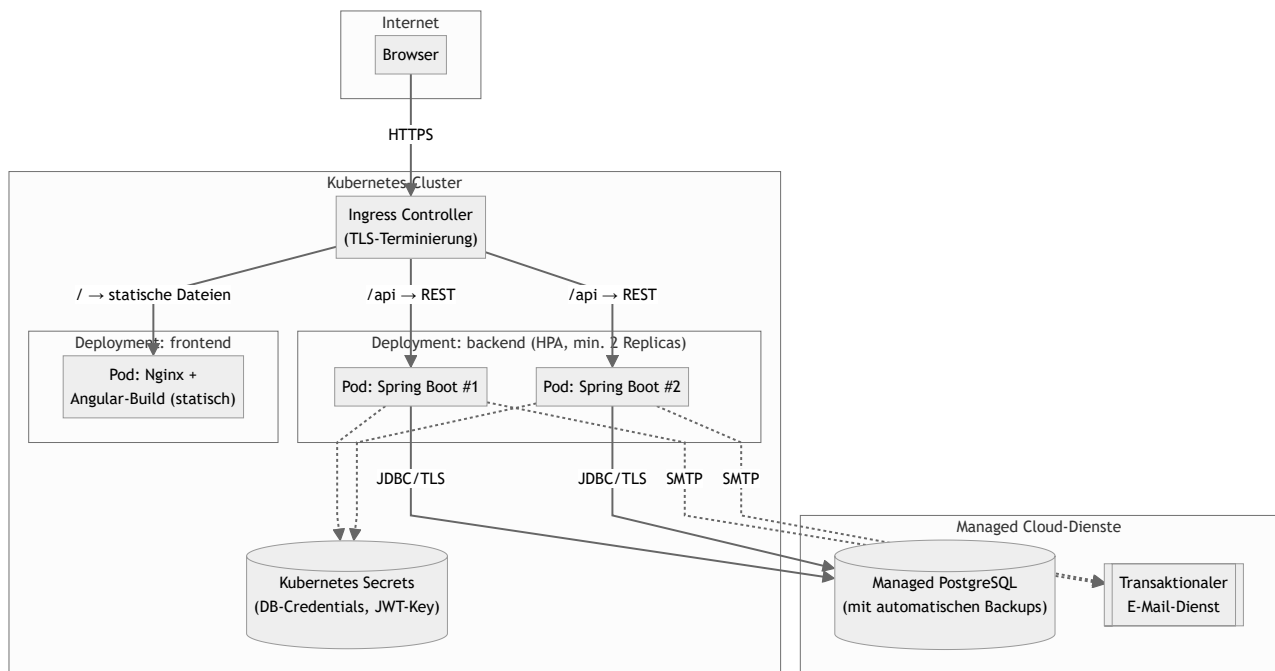


6.5 Preisabfrage



7. Verteilungssicht

Cloud-natives Deployment auf einem Kubernetes-Cluster (Anbieter austauschbar, z. B. AWS EKS, Azure AKS, oder Hetzner + k3s für Kosteneffizienz im Portfolio-Kontext):



Begründungen:

- **Managed PostgreSQL** statt Self-Hosting in Kubernetes: automatische Backups, Patching und Hochverfügbarkeit werden nicht selbst betrieben (relevant bei einem Ein-Personen-Team).
- **Horizontal Pod Autoscaler (HPA)** für das Backend: Lastspitzen zu Schichtwechselzeiten (Stempeln) oder bei filialübergreifenden Inventuren werden abgefangen.
- **Statisches Frontend** wird als reine Build-Artefakte über Nginx (oder alternativ CDN) ausgeliefert — kein Server-Side-Rendering nötig.
- **CI/CD:** Pipeline (z. B. GitHub Actions) baut Docker-Images für `frontend` und `backend`, pusht sie in eine Container-Registry und deployed via `kubectl` / Helm auf den Cluster.

8. Querschnittliche Konzepte

Sicherheit

- Passwort-Hashing mit BCrypt, niemals Klartext-Speicherung.
- JWT Access-Token (kurzlebig, ~15 min) + Refresh-Token (langlebig, revozierbar in DB) — begrenzt Schaden bei Token-Diebstahl.
- Rollenbasierte Autorisierung via `@PreAuthorize` auf Service-/Controller-Ebene (Mitarbeiter, Filialleiter, HR, Admin).
- Filial-Scoping: Filialleiter dürfen nur Daten der eigenen Filiale sehen/ändern (zusätzlich zur Rollenprüfung eine Zugehörigkeitsprüfung).
- Durchgängig HTTPS, keine sensiblen Daten in URLs/Logs.

Persistenz

- Spring Data JPA für Zugriff, aber Flyway-Migrationsskripte als alleinige Quelle der Wahrheit für das Schema (kein `ddl-auto: update` in Produktion).

- Zeitstempel durchgängig in UTC (`timestamp tz`), Umrechnung auf lokale Zeitzone (Europe/Vienna) erst im Frontend.

Nachvollziehbarkeit / Audit-Trail

- Für Zeitkorrekturen und Abschreibungen wird jede Statusänderung mit Zeitstempel, ausführendem Benutzer und Grund protokolliert (eigene Audit-Tabellen statt nachträglicher Log-Analyse).
- Ursprüngliche Werte bleiben erhalten (kein Überschreiben), Korrekturen erzeugen neue, referenzierte Datensätze.

Fehlerbehandlung

- Einheitliches Fehlerformat nach RFC 7807 (Problem Details), zentral über `@ControllerAdvice`.
- Fachliche Fehler (z. B. „Korrektur bereits genehmigt“) vs. technische Fehler (500er) werden unterschieden und unterschiedlich geloggt.

Validierung

- Eingabevalidierung über Bean Validation (`jakarta.validation`) auf DTO-Ebene, zusätzlich fachliche Prüfungen im Service (z. B. keine negative Lagerbestandsmenge).

Logging & Monitoring

- Strukturiertes JSON-Logging, Correlation-ID pro Request (Nachvollziehbarkeit über Pod-Grenzen hinweg).
- Health-/Readiness-Endpunkte (Spring Boot Actuator) für Kubernetes-Probes.

API-Design

- REST über JSON, OpenAPI-Dokumentation via springdoc-openapi (automatisch generiert, dient auch als Frontend-Vertrag).

Testkonzept

- Backend: Unit-Tests (JUnit 5, Mockito), Integrationstests mit Testcontainers (echte PostgreSQL-Instanz statt H2, um Flyway-Migrationen realistisch zu testen).
- Frontend: Unit-Tests mit Vitest.
- Kritische Abläufe (Zeitkorrektur-Workflow, Inventur-Abschluss) zusätzlich über Komponenten-/Integrationstests abgedeckt.

Internationalisierung

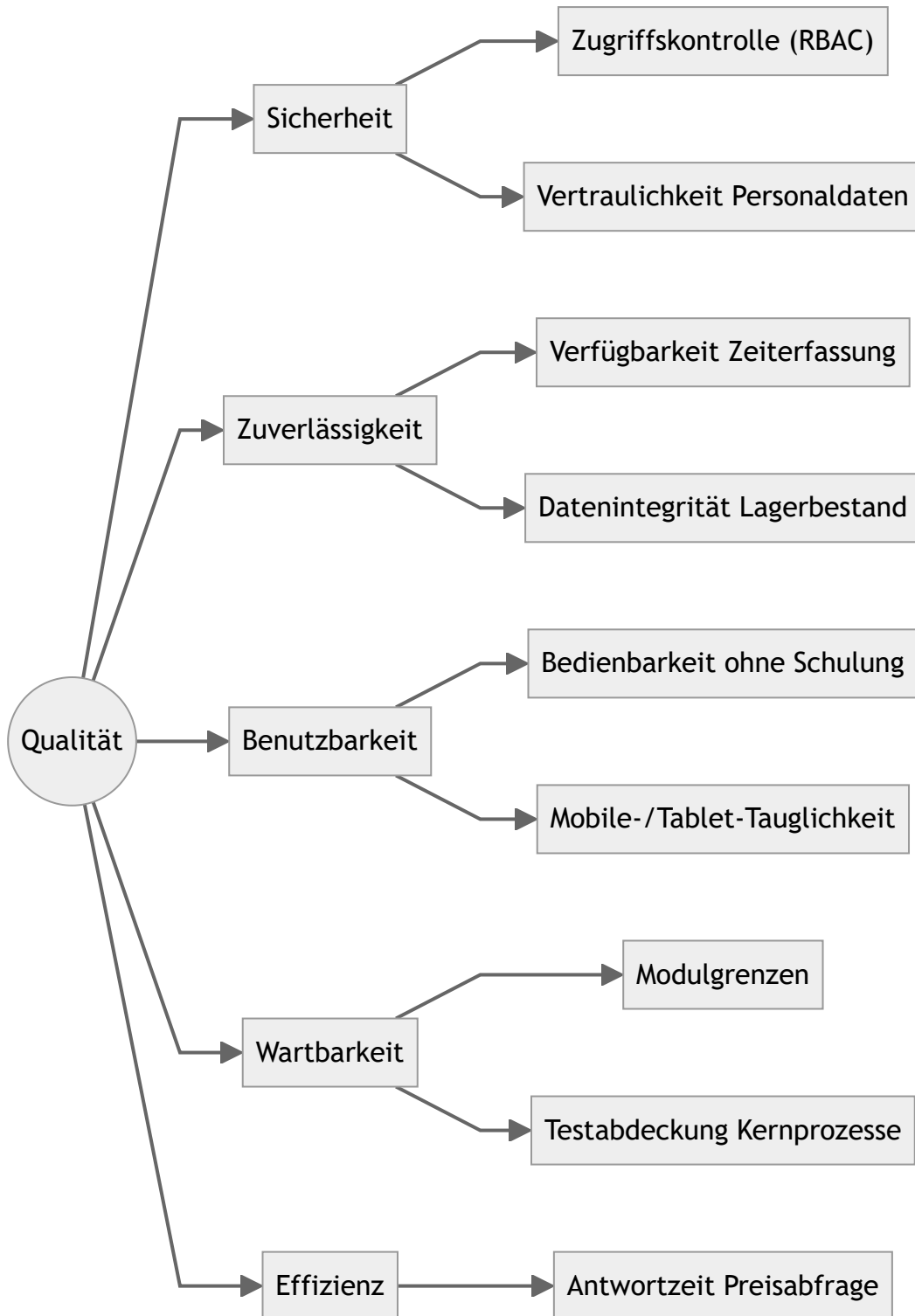
- UI-Texte primär Deutsch, aber von Anfang an über Angular-i18n-Mechanismus ausgelagert, um spätere Mehrsprachigkeit ohne Refactoring zu ermöglichen.

9. Architekturentscheidungen

ID	Entscheidung	Kontext	Konsequenz
ADR-001	Modularer Monolith statt Microservices	Ein-Personen-Team, überschaubare Domäne	Geringerer Betriebsaufwand; Modulgrenzen im Code (Packages) ermöglichen späteren Schnitt in Services
ADR-002	PostgreSQL als einzige Datenbank	ACID-Anforderungen bei Arbeitszeiten/Lagerbeständen, bereits im Grundgerüst vorgegeben	Kein Polyglot Persistence, dafür starke Konsistenzgarantien
ADR-003	JWT statt serverseitiger Sessions	Horizontale Skalierung mehrerer Backend-Pods ohne Session-Replikation	Erfordert Refresh-/Revocation-Konzept für Logout und Sicherheit
ADR-004	Ein Firmenmodell mit mehreren Filialen (kein Multi-Tenant)	Anforderung: interne Lösung für ein Unternehmen	Einfacheres Datenmodell; Umbau nötig, falls später mehrere Kundenfirmen bedient werden sollen
ADR-005	Flyway für Schema-Migrationen	Nachvollziehbare, versionierte DB-Änderungen im Team-/Solo-Kontext	Jede Schemaänderung erfordert ein Migrationsskript, kein spontanes Auto-DDL
ADR-006	Kubernetes als Deployment-Ziel	Zukunftsfähigkeit, Skalierbarkeit bei Lastspitzen	Höherer initialer Infrastruktur-Aufwand als z. B. einfaches Docker Compose
ADR-007	Angular Standalone Components (kein NgModule-basierter Aufbau)	Aktueller Angular-Standard (v22), bereits im Grundgerüst so angelegt	Schlankere Struktur, aber Team muss mit neuerem Angular-Stil vertraut sein

10. Qualitätsanforderungen

10.1 Qualitätsbaum (angelehnt an ISO 25010)



10.2 Qualitätsszenarien

#	Qualitätsmerkmal	Szenario
1	Sicherheit	Ein Mitarbeiter versucht, per direktem API-Aufruf Zeitkorrekturen einer anderen Filiale einzusehen → das System lehnt mit HTTP 403 ab, der Versuch wird geloggt.
2	Nachvollziehbarkeit	Ein Filialleiter genehmigt eine Zeitkorrektur → im System bleibt nachvollziehbar, wer, wann, mit welcher Begründung genehmigt hat, inklusive des ursprünglichen Werts.

#	Qualitätsmerkmal	Szenario
3	Verfügbarkeit	Ein Backend-Pod fällt während des Betriebs aus → Requests werden durch verbleibende Replicas ohne Nutzerunterbrechung weiterverarbeitet (Antwortzeit-Ziel: < 2 s Ausfallfenster).
4	Performance	Eine Preisabfrage per Barcode liefert bei normaler Systemlast innerhalb von 300 ms eine Antwort.
5	Benutzbarkeit	Ein neuer Mitarbeiter kann ohne Einschulung „Kommen“ buchen (max. 2 Klicks/Taps ab Login).
6	Datenintegrität	Zwei Mitarbeiter erfassen gleichzeitig einen Inventur-Zählwert für denselben Artikel → das System verhindert widersprüchliche Endzustände (transaktionale Konsistenz).
7	Wartbarkeit	Ein neues Modul „Urlaubsverwaltung“ soll ergänzt werden → dies ist möglich, ohne bestehende Module Personalmanagement/Warenwirtschaft anzupassen.
8	Skalierbarkeit	Am Monatsersten (hohe Zeiterfassungs-Last durch viele gleichzeitige Buchungen) skaliert das Backend automatisch auf zusätzliche Pods.

11. Risiken und technische Schulden

Risiko / Schuld	Beschreibung	Umgang
Kein Multi-Tenant-Modell	Falls später mehrere Kundenfirmen bedient werden sollen, ist ein größerer Umbau des Datenmodells nötig (Mandanten-Trennung)	Bewusst akzeptiert, da aktuelle Anforderung eine interne Lösung ist (siehe ADR-004)
Kein Zwei-Faktor-Login	Trotz sensibler Personaldaten aktuell nur Passwort + JWT	Für spätere Ausbaustufe vorgemerkt, insbesondere für Admin-/HR-Rollen
Kubernetes-Komplexität	K8s-Setup ist für ein Portfolio-Projekt initial aufwändig (Cluster, Ingress, Secrets, HPA)	Für die ersten Entwicklungsphasen kann lokal/Staging mit Docker Compose gearbeitet werden; K8s-Manifeste werden parallel aufgebaut
Bus-Faktor 1	Einzelentwickler-Projekt — Wissen ist nicht verteilt	Durch dieses arc42-Dokument und Code-Kommentare an nicht-offensichtlichen Stellen wird Wissen dauerhaft festgehalten
Keine Offline-Fähigkeit	Zeiterfassung benötigt aktuell durchgängige Internetverbindung der Filiale	Für MVP akzeptiert; Offline-First (z. B. Service Worker + lokale Queue) als mögliche spätere Erweiterung
DSGVO-Detailkonzept offen	Löschfristen, Aufbewahrungspflichten (insb. für Lohn-relevante Zeiterfassungsdaten) sind noch nicht im Detail spezifiziert	Muss vor Produktivbetrieb mit realen Personaldaten ausgearbeitet werden
Externe Schnittstellen (Lohnsystem, Kassensystem) nicht spezifiziert	Aktuell nur als zukünftige Nachbarsysteme dokumentiert (Kapitel 3), keine konkrete Schnittstelle definiert	Wird bei Bedarf in einem eigenen Schnittstellen-ADR nachgezogen

12. Glossar

Begriff	Definition
Firma	Das eine Unternehmen, das dieses System betreibt (kein Multi-Tenant-Konzept)
Filiale	Standort/Verkaufsstelle der Firma; Bezugspunkt für Mitarbeiter-Zuordnung und Lagerbestände
Mitarbeiter	Angestellte Person, einer Filiale zugeordnet; Fachobjekt im Modul Personalmanagement
Filialleiter	Rolle mit erweiterten Rechten innerhalb der eigenen Filiale (Zeiten korrigieren/genehmigen, Inventur/Abschreibung)
Arbeitszeiteintrag	Einzelne Stempelung (Kommen/Gehen) eines Mitarbeiters
Korrekturantrag	Antrag auf nachträgliche Änderung eines Arbeitszeiteintrags, durchläuft einen Genehmigungsworkflow
Artikel	Ware im Sortiment, identifiziert u. a. per Barcode/EAN
Lagerbestand	Aktuell erfasste Menge eines Artikels in einer bestimmten Filiale
Inventur	Periodischer Soll-/Ist-Abgleich der Lagerbestände einer Filiale
Abschreibung	Buchung eines Warenverlusts (Verderb, Bruch, Diebstahl) mit Mengen- und Wertangabe
Preisabfrage	Schnelle Ermittlung von Artikelpreis/-bezeichnung, typischerweise per Barcode-Scan
RBAC	Role-Based Access Control — rollenbasierte Zugriffssteuerung
JWT	JSON Web Token — zustandsloses Token-Format zur Authentifizierung
Bounded Context	Fachlich abgegrenzter Modellbereich (hier: Personalmanagement, Warenwirtschaft, Stammdaten)
HPA	Horizontal Pod Autoscaler — Kubernetes-Mechanismus zur automatischen Skalierung von Pods